

Receipt Line Items for Anonymity Claims: A Minimal Tuple Schema for Published Budgets (Series Synthesis)

Unified archive note v0.31 (2026-03-05; archive v162)

Abstract

Many papers in this archive culminate in a short published object: a label (OINL), a budget (e.g., MaxL), and a receipt that pins the claim to concrete knobs under change control. (Receipt envelopes are specified by ABOM/OINL in Anonymity C; this note standardizes the line-item tuples carried inside.)[3]

To keep the series non-redundant and referee-grade, we centralize the receipt-facing interface: *what minimal tuple must appear as a line item so that a reader can understand the claim and an auditor can request exactly the missing artifacts?* We give a small schema that (i) is stable across modules (retries, congestion, destination equalization, state/auditing), and (ii) cleanly separates *budget semantics* from *publication mechanics*.

Scope. This is a short schema/interface note. It does not propose new mechanisms; it standardizes *what to publish* when a mechanism paper exports a budget.

Imports. Contract-object vocabulary is imported from Synthesis 0.[2] ABOM/OINL packaging is imported from Anonymity C.[3] Transparency primitives (beacon/VRF binding, append-only logs, inclusion/consistency proofs) are imported from Synthesis 9.[4] Endpoint metric semantics (odds inflation / MaxL and conversions) are imported from Synthesis 5 and Anonymity A.[5, 6] Replay-plan and state-declaration drift/equivalence rules are imported from Synthesis 18.[9]

Exports. A minimal, receipt-ready *line item tuple* and a “who owns what” map so other papers can remain tight.

1 Receipt envelope (three IDs + optional pointers)

A receipt is a small published object intended to be stable across modules. To keep replay and drift semantics non-redundant, we separate the *envelope* (series-wide identifiers) from per-claim line items.

Definition 1 (Receipt envelope). *A receipt begins with:*

- *claim_id*: a stable identifier for the claim family (often a digest ID);
- *tw_id*: the threat-window declaration governing repetitions/resets;
- *state_decl_id* (optional): scoped state/measurement declaration used during evaluation; include when long-lived state or monitored quantities enter the guarantee.

Identifier and digest conventions are centralized in Synthesis 8; plan/state equivalence is centralized in Synthesis 18.[27, 9]

Remark 1 (Why the envelope matters). Most “anonymity bugs” in deployed systems are drift bugs: the receipt is still published, but the knobs/state/measurement surface has changed. Separating the envelope makes it explicit what must be re-issued (new `state_decl_id` or `tw_id`) versus what may be replayed (same `plan_spec_id` under Synthesis 18).

Auditor replay. This note defines the tuple surface; the canonical mechanical replay checklist (anchoring $\rightarrow$ bundle fetch $\rightarrow$ to-spec replay) is Synthesis 20.[10]

2 The line item (definition)

Definition 2 (Receipt line item). A receipt line item is a short, hash-addressable record that binds a budget claim to: (i) a declared witness surface (transcript tier/projection), (ii) a declared repetition window and usage assumption, and (iii) the specific knobs/parameters whose drift would change the claim. The item is designed so an auditor can replay the accountant given access to the artifact bundle referenced by ABOM/OINL.

Contact surfaces are semantic. When a line item names an external service contact surface (e.g., delegated routing or a fallback committee), the contact surface is a canonical description of the interface: *method*, full URL (including path scope), and any declared request-shaping defaults (e.g., IPIP-0484 protocol filtering) and time budgets that affect who is contacted and how termination-time behaves. Changing host, path, method, default filter parameters, or declared time budgets is interface drift and triggers recertification. [29, 9, 26]

Remark 2. For publication and user micro-diff hygiene, we recommend publishing `contact_surface_id` as `sha256(JCS(contact_surface))` (`JCS = RFC 8785`)[1] so the interface can be compared atomically across releases.

3 Minimal tuple schema (stable across modules)

We recommend the following minimal fields. (Names are schematic; deployments may choose concrete encodings, but the semantics should match.)

Remark 3 (Evidence versus replay). `evidence_id` names what supports the number in `budget_value` (a log-audit table, a dual certificate, a proof transcript, ...). `replay_hook` names how an auditor should recompute and validate it (a stable `plan_id` with a digest-bound `plan_spec_id`, plus the associated bundle pointer on request). They are intentionally separate: a single replay plan may emit multiple evidence objects, and an evidence object may be checkable by more than one replay plan.

Remark 4 (Separation-class primary paths need an explicit guard). Some deployments publish a primary-path line item whose numeric value is interpreted only under a role-separation assumption (e.g., “0 bits on a linkability surface under `sep.nr1`”). In that case, the line item should commit to a separation-manifest digest (Synthesis 22) inside `obs_model` or `knobs` so the non-collusion assumption and invalidators are diffable. If the deployment is willing to publish an upper bound ρ on separation failure within the threat window, carry it as `rho_upper` in the same place and interpret it as the guarded checkpoint $(0, \rho)$ per Synthesis 23. Tightening `rho_upper` should be treated

as *replay/evidence-required*; relaxing it may be tagged as *lineage=coarsening* but still must be published and diffed.[20, 21]

Remark 5 (Why this is the right granularity). *The tuple intentionally stops short of shipping full artifacts: it makes the claim addressable. A reader can compare releases from the public tuple alone, while an auditor can request the precise evidence referenced by `replay_hook`. This matches the source-only artifact policy.[24] If deployments want end users to discover the right receipt and mechanically validate its anchoring, pair the tuple with the minimal user-verifiability record in Synthesis 13.[19]*

Remark 6 (Interface ids and lineage prevent accidental claim reuse). *A line item should never rely on prose like “same mechanism as last release.” The pair (`exposure_nf_id`, `lineage`) makes the intended relation explicit: if the interface is unchanged, the new receipt says so; if the interface is an explicit coarsening, the receipt says so; otherwise the receipt declares a new interface and stops pretending the old certificate applies automatically. This keeps release engineering, recertification, and user-facing validation aligned.*

Replay workflows and state assumptions have the same “label drift” failure mode: a bare plan name or a human state label is not a stable audit promise. Receipts should therefore carry digest-bound `plan_spec_id` and (when relevant) `state_decl_id`, and treat changes to these ids using the equivalence/drift taxonomy centralized in Synthesis 18.[9]

4 Module-specific instantiations (short)

This note intentionally avoids module-specific prose. If a mechanism needs extra fields beyond Table 1, publish them under a single `module_ext` map and bind any semantic interfaces (e.g., service contact surfaces) by digest ids such as `contact_surface_id`. A module that exports a receipt line item should only specify what it *owns* (the meaning of `usage`, `obs_model`, `knobs`, `budget_type`, and any evidence objects) and cite this note for the tuple shape. For an end-to-end instantiation (including the interplay with state declarations and deployed routing knobs), see the worked example (Synthesis 17) and the deployed-surface note (Synthesis 30).[23, 29]

5 A publication checklist (one paragraph)

To publish a line item responsibly: (i) include Table 1’s fields; (ii) include a stable `exposure_nf_id` and, when applicable, a `lineage` relation to the previous claim; (iii) commit the tuple to ABOM/OINL and to a release receipt;[3, 25] (iv) anchor it in a transparency log with inclusion/consistency proofs;[4] and (v) treat any knob drift or interface drift as a recertification trigger (Certified C).[26] When public discovery matters, publish a companion UVI record that points users to the anchored receipt tuple and its validation path.[19] For a concrete end-to-end worked instantiation of these fields, see the worked-example micro-note (Synthesis 17).[23]

References

- [1] A. Rundgren. JSON Canonicalization Scheme (JCS). RFC 8785, IETF, 2020.
- [2] Anonymous. Contract Objects for Anonymous-DHT Privacy Budgets and Claims. Synthesis 0, The-One-True-Archive (2026).

- [3] Anonymous. ABOM and OINL for Anonymity Claims. Anonymity series Paper C, The-One-True-Archive (2026).
- [4] Anonymous. Transparency Primitives for Privacy Receipts: Beacons, VRFs, and Append-Only Logs. Synthesis 9, The-One-True-Archive (2026).
- [5] Anonymous. Endpoint Metrics Bridge: Odds Inflation, PML/MaxL, and Identification Sizing. Synthesis 5, The-One-True-Archive (2026).
- [6] Anonymous. Odds Inflation, Privacy Loss, and Endpoint Anonymity Bounds. Anonymity series Paper A, The-One-True-Archive (2026).
- [7] Anonymous. Trace Interfaces for Anonymous DHTs. Synthesis 3, The-One-True-Archive (2026).
- [8] Anonymous. Threat Models and Repetition Windows for Anonymous DHTs. Synthesis 4, The-One-True-Archive (2026).
- [9] Anonymous. Digest-Bound Replay Plans and State Declarations: Equivalence, Minimality, and Change Control. Synthesis 18, The-One-True-Archive (2026).
- [10] Anonymous. Auditor Replay for Anonymous-DHT Receipt Line Items: A Mechanical Checklist for Digest-Bound Claims. Series synthesis note (Synthesis 20), 2026. (This archive.)
- [11] Anonymous. Adaptive Fallback as a Witness Stream: The Path-Selection Tax and Honest Tiered Receipts. Series synthesis note (Synthesis 14), The-One-True-Archive (2026).
- [12] Anonymous. Observation Attenuation for Repeated-Use Privacy Budgets: Mixture/Subsampling Bounds and a Stub-Mix Interface. Synthesis 11, The-One-True-Archive (2026).
- [13] Anonymous. Tiered Observation Vectors for Anonymous-DHT Receipts: Contact-Set, Timing, and Congestion Tiers with Conservative MaxL Composition. Synthesis 15, The-One-True-Archive (2026).
- [14] Anonymous. AnonDHT in the Boss Fight Regime: Compiling a MaxL Budget into Lookup Knobs. Boss Fight series Paper B, 2026. (This archive.)
- [15] Anonymous. MC-EQ and CoverSketch: Destination Privacy via Markov-Chain Equalization in Anonymous DHT Lookups. Release & destination Paper B, The-One-True-Archive (2026).
- [16] Anonymous. W-Congestion-EQ: Verifiable Congestion Budgets (Certificates + Receipts). Congestion series Paper 3, The-One-True-Archive (2026).
- [17] Anonymous. Optimal Poset-Feasible Padding: Knapsack, Transport, and Dual Certificates for TV Privacy. Published backbone note (2026-01-29). Wiki: `[[2026.01.29 - Mathematics: Optimal Poset-Feasible Padding: Knapsack, Transport, and Dual Certificates for TV Privacy]]`.
- [18] Anonymous. Closed-View Auditing and CPPC Manifests. AnonDHT state series Paper 3, The-One-True-Archive (2026).
- [19] Anonymous. User-Verifiable Receipt Interfaces for Anonymous-DHT Claims: A Minimal Discoverability and Validation Contract. Synthesis 13, The-One-True-Archive (2026).
- [20] Anonymous. Primary-Path Separation Manifests: Guard Objects for Non-Collusion Assumptions in Reader-Privacy Receipts. Synthesis 22, The-One-True-Archive (2026).
- [21] Anonymous. Probabilistic Separation Budgets: Separation Failure as a Receipt-Grade (ϵ, δ) Adjunct. Synthesis 23, The-One-True-Archive (2026).
- [22] Anonymous. Conditional Per-Step Budget Certificates: A Receipt-Grade Interface for Adaptive Horizon Claims. Synthesis 21, The-One-True-Archive (2026).

- [23] Anonymous. Worked Example: Instantiating a Tiered Reader-Privacy Receipt. Synthesis 17, The-One-True-Archive (2026).
- [24] Anonymous. Artifact and Disclosure Policy for the One-True-Archive (Source-Only Public Release). Synthesis 6, The-One-True-Archive (2026).
- [25] Anonymous. Privacy as a Release Property: Proof-Carrying Anonymity Receipts for Anonymous DHTs. Release & destination Paper A, The-One-True-Archive (2026).
- [26] Anonymous. Disagreement-Gated Recertification. Certified series Paper C, The-One-True-Archive (2026).
- [27] Anonymous. Notation and Minimal Model for the One-True-Archive. Synthesis 8, The-One-True-Archive (2026).
- [28] Anonymous. State-Dependent Anonymity: Selection and State Evolution as an Attack Surface. AnonDHT State series Paper 1, The-One-True-Archive (2026).
- [29] Anonymous. Deployed Observation Surfaces for Anonymous DHTs: Control-plane knobs in IPFS/libp2p delegated routing and OHTTP evolution. Series synthesis note (Synthesis 30), 2026. (This archive.)

Field	Meaning (and who owns the semantics)
<code>claim_id</code>	Stable identifier for the claim object (ABOM/OINL carrier).[3]
<code>exposure_nf_id</code>	Stable exposure normal-form id / schema hash for the projection being budgeted. This is the real interface identifier; the rest of the tuple refines it.[7]
<code>witness_tier</code> (optional)	Human-readable tier label for quick reading; redundant to <code>exposure_nf_id</code> and should be treated as non-binding.[7]
<code>tw_id</code>	Threat-window declaration: what repetition horizon the claim is sized for.[8]
<code>state_decl_id</code> (optional)	Digest-bound state-surface declaration committing to which long-lived control-plane state surfaces are in scope for the bound (and which are excluded). This includes deployed router sets, path scope, request-shaping defaults (e.g., filters), timeout/refresh cadences, downgrade/fallback rules, and retrieval-mode switches when they affect the transcript.[28, 29, 23]
<code>contact_surface_id</code> (optional)	Digest-bound id for a named external service contact surface (method + URL + request-shaping defaults + time budgets). Recommended: <code>sha256(JCS(contact_surface))</code> . [1]
<code>evidence_id</code> (optional)	Digest-bound identifier for a concrete evidence object (log audit table, dual certificate, proof transcript, ...) supporting <code>budget_value</code> . Naming conventions live in Synthesis 8; conditional/per-step evidence objects are standardized in Synthesis 21.[27, 22]
<code>usage</code>	Usage assumption inside the window (e.g., Q lookups/window; rate-limit policy).[14]
<code>budget_type</code>	What is being bounded (e.g., MaxL in bits, OINL odds inflation, or a TV certificate).[5, 17]
<code>budget_value</code>	The bound value (e.g., b bits per lookup, or an identified OINL label).[6, 3]
<code>obs_model</code>	Observation parameterization (e.g., p_{obs} , a monitored upper bound, a tiered observation vector, or tiered path-selection bounds for fallback use).[12, 13, 11]
<code>knobs</code>	Hashes/values of the knobs that determine the accountant (retries, timeouts, widths, calendars, equalization params).[14, 15]
<code>lineage</code>	Optional predecessor pointer plus relation tag (<code>same-enf</code> , <code>coarsening</code> , or <code>new-interface</code>). This is what makes cross-release reuse explicit instead of implicit.[7, 26, 25]
<code>replay_hook</code>	How an auditor replays the accountant: a stable <code>plan_id</code> plus a digest-bound <code>plan_spec_id</code> (canonical plan specification hash) and an artifact-bundle pointer (on request).[2, 4]

Table 1: A minimal receipt line-item schema. Papers should avoid restating semantics for these fields; instead, cite the owning note listed in the right column.